

Guía de contenidos: RECORD en PL/SQL

Clase 1.1.1 · Indicador de logro 1.1 · Modelo de datos BANK SOLUTIONS

Asignatura	BDY1103 – Taller de Base de Datos	Experiencia	EA1
Tiempo	1 h 20 min (clase) + trabajo autónomo	Modalidad	Individual
Herramienta	Oracle SQL Developer	Base de datos	BANK SOLUTIONS

1. El problema que resuelve un RECORD

Hasta ahora has trabajado con variables escalares: cada variable guarda un solo valor. Eso funciona bien cuando necesitas un dato, pero se vuelve incómodo apenas necesitas describir una entidad completa.

Mira este bloque, que obtiene la ficha de un cliente de BANK SOLUTIONS:

```
DECLARE
  v_run      VARCHAR2(15);
  v_pnombre  CLIENTE.pnombre%TYPE;
  v_appaterno CLIENTE.appaterno%TYPE;
  v_direccion CLIENTE.direccion%TYPE;
  v_inscrip  CLIENTE.fecha_inscripcion%TYPE;
BEGIN
  SELECT TO_CHAR(numrun)||'-'||dvrn, pnombre, appaterno, direccion, fecha_inscripcion
  INTO   v_run, v_pnombre, v_appaterno, v_direccion, v_inscrip
  FROM   CLIENTE
  WHERE  nro_cliente = 3;
END;
/
```

Son cinco variables sueltas que, en realidad, describen a una sola persona. Si mañana la ficha necesita dieciocho datos, tendrás dieciocho variables. Y si quisieras pasar «el cliente» a otro trozo de programa, tendrías que arrastrarlas todas.

Un RECORD resuelve exactamente esto: agrupa varios datos relacionados, de distinto tipo, bajo un solo nombre de variable.

Una variable escalar es una casilla con un rótulo. Un RECORD es una ficha de papel con varias casillas rotuladas dentro. La ficha tiene un nombre («ficha del cliente») y cada casilla también («RUN», «nombre», «dirección»). Para llegar a una casilla, primero nombras la ficha y después la casilla.

2. Dónde encaja RECORD entre los tipos de PL/SQL

Categoría	Qué guarda	Ejemplos
Escalar	Un único valor sin estructura interna	NUMBER, VARCHAR2, DATE, BOOLEAN, CHAR
Compuesto	Varios valores accesibles por nombre o por índice	RECORD, VARRAY, tablas anidadas, tablas indexadas
Referencia	Un puntero a otra estructura	REF CURSOR, REF tipo_objeto
LOB	Grandes volúmenes de datos	BLOB, CLOB, NCLOB, BFILE

RECORD y VARRAY son ambos tipos compuestos, pero resuelven problemas distintos: **un RECORD agrupa campos diferentes entre sí** (nombre, edad, dirección); un VARRAY guarda **muchos elementos del mismo tipo** (una lista de códigos, una lista de montos). Los verás combinados más adelante.

3. Declarar un RECORD

Trabajar con un RECORD siempre tiene dos pasos, y confundirlos es el error más frecuente al empezar.

Paso 1: definir el tipo (el molde)

```
TYPE nombre_del_tipo IS RECORD (  
    campo1 tipo_de_dato [NOT NULL] [:= valor_inicial],  
    campo2 tipo_de_dato [NOT NULL] [:= valor_inicial],  
    ...  
);
```

Esta línea no crea ninguna variable. Solo describe cómo será la estructura. El tipo existe únicamente dentro del bloque, subprograma o package donde se declara.

Paso 2: declarar la variable (la pieza fabricada con el molde)

```
nombre_variable nombre_del_tipo;
```

Recién aquí nace la variable, y nace con todos sus campos en NULL.

Ejemplo

```
DECLARE  
    -- 1. El molde  
    TYPE t_credito IS RECORD (  
        codigo    CREDITO.cod_credito%TYPE,  
        nombre     CREDITO.nombre_credito%TYPE,  
        tasa       NUMBER(4,2),  
        cuotas     NUMBER(3) := 12          -- valor por defecto  
    );  
  
    -- 2. La variable  
    v_credito t_credito;  
BEGIN  
    v_credito.codigo := 2;  
    v_credito.nombre := 'Crédito de Consumo';  
    v_credito.tasa    := 14.5;  
  
    DBMS_OUTPUT.PUT_LINE(v_credito.nombre || ' - tasa ' || v_credito.tasa || '%');  
END;  
/
```

Regla del punto: para leer o escribir un campo se usa variable.campo. Nunca tipo.campo. Escribir t_credito.nombre es un error de compilación.

Campos con NOT NULL

Si declaras un campo como NOT NULL, PL/SQL exige que le des un valor por defecto en la misma declaración. De lo contrario la variable nacería NULL, lo que contradice la restricción.

```
TYPE t_cuota IS RECORD (  
    nro_cuota NUMBER(2) NOT NULL := 1,    -- correcto  
    valor     NUMBER(10)                -- correcto (admite NULL)  
);  
  
TYPE t_malo IS RECORD (  
    nro_cuota NUMBER(2) NOT NULL          -- PLS-00218: falta la inicialización  
);
```

4. Cargar un RECORD desde la base de datos

La forma habitual de llenar un RECORD no es campo por campo, sino de una sola vez con SELECT ... INTO.

```
DECLARE
  TYPE t_ficha IS RECORD (
    run          VARCHAR2(15),
    nombre       VARCHAR2(120),
    direccion    CLIENTE.direccion%TYPE
  );
  v_ficha t_ficha;
BEGIN
  SELECT TO_CHAR(numrun) || '-' || dvrn,
         TRIM(pnombre || ' ' || appaterno),
         direccion
  INTO   v_ficha
  FROM   CLIENTE
  WHERE  nro_cliente = 3;

  DBMS_OUTPUT.PUT_LINE(v_ficha.run || ' | ' || v_ficha.nombre);
END;
/
```

El punto donde más se falla: la correspondencia entre las columnas del SELECT y los campos del RECORD es POSICIONAL, no por nombre. La primera columna va al primer campo, la segunda al segundo, y así. Si inviertes dos columnas, o el bloque falla con ORA-06502 o —peor— guarda los datos cruzados sin avisar.

También puedes cargar un campo específico, lo que resulta útil cuando el RECORD se llena desde varias consultas:

```
SELECT COUNT(*) INTO v_ficha.total_credits
FROM   CREDITO_CLIENTE WHERE nro_cliente = 3;
```

5. El atributo %ROWTYPE

Cuando el RECORD que necesitas tiene exactamente la misma estructura que una tabla, no hace falta escribirlo campo por campo. El atributo %ROWTYPE lo genera por ti:

```
DECLARE
  v_credito CREDITO%ROWTYPE;  -- un campo por cada columna de CREDITO,
                                -- con el mismo nombre y el mismo tipo
BEGIN
  SELECT * INTO v_credito
  FROM   CREDITO
  WHERE  cod_credito = 2;

  DBMS_OUTPUT.PUT_LINE(v_credito.nombre_credito);
  DBMS_OUTPUT.PUT_LINE('Tasa anual: ' || v_credito.tasa_interes_anual || '%');
END;
/
```

Los nombres de los campos son los nombres de las columnas. Y si mañana se agrega una columna a CREDITO, tu bloque se adapta solo al recompilarse.

También funciona sobre un cursor, algo que usarás mucho en la sesión de cursores explícitos:

```
CURSOR c_clientes IS SELECT nro_cliente, pnombre, appaterno FROM CLIENTE;
v_filas c_clientes%ROWTYPE;  -- tres campos: nro_cliente, pnombre, appaterno
```

%TYPE, %ROWTYPE y RECORD explícito: cuándo usar cada uno

	%TYPE	%ROWTYPE	RECORD explícito
Qué declara	Una variable escalar	Una variable compuesta	Una variable compuesta
Estructura	El tipo de una columna	Todas las columnas de una tabla, vista o cursor	Exactamente los campos que tú definas
Ejemplo	v_run CLIENTE.numrun%TYPE;	v_cli CLIENTE%ROWTYPE;	TYPE t_x IS RECORD (...); v_x t_x;
Se adapta si cambia la tabla	Sí	Sí	Solo los campos declarados con %TYPE
Permite campos calculados	No aplica	No	Sí
Permite mezclar tablas	No aplica	No	Sí
Úsalo cuando...	Necesitas un solo dato de una columna	Necesitas la fila completa tal cual está en la tabla	Necesitas un subconjunto, un cálculo o datos de varias tablas

6. Operaciones entre RECORD

Asignación completa

Puedes copiar un RECORD entero en otro, siempre que ambos sean del mismo tipo:

```
DECLARE
  v_original  CLIENTE%ROWTYPE;
  v_respaldo  CLIENTE%ROWTYPE;
BEGIN
  SELECT * INTO v_original FROM CLIENTE WHERE nro_cliente = 1;
  v_respaldo := v_original;      -- copia los 18 campos de una vez
END;
/
```

Cuidado: dos tipos RECORD con exactamente los mismos campos siguen siendo tipos distintos. Asignar entre ellos produce PLS-00382. En PL/SQL la equivalencia de estructura no es equivalencia de tipo.

Comparación

No puedes comparar dos RECORD con =, <> ni con IS NULL. Hay que comparar campo por campo:

```
-- IF v_a = v_b THEN          -- error de compilación
IF v_a.codigo = v_b.codigo AND v_a.nombre = v_b.nombre THEN
  ...
END IF;
```

RECORD anidados

Un campo de un RECORD puede ser, a su vez, otro RECORD. Esto sirve para agrupar información por bloques lógicos:

```
DECLARE
  TYPE t_resumen_cuotas IS RECORD (
    pagadas      NUMBER(3),
    monto        NUMBER(12),
    pendiente    NUMBER(12)
  );

  TYPE t_estado_credito IS RECORD (
    nro_solicitud CREDITO_CLIENTE.nro_solic_credito%TYPE,
    nombre        CREDITO.nombre_credito%TYPE,
    cuotas        t_resumen_cuotas          -- un RECORD dentro de otro
  );

  v_estado t_estado_credito;
BEGIN
  v_estado.cuotas.pagadas := 8;      -- se encadenan los puntos
END;
/
```

Restricción importante: un SELECT ... INTO no puede cargar de una vez un RECORD que contiene otro RECORD. Debes cargar los campos por separado, indicando la ruta completa: INTO v_estado.cuotas.pagadas, v_estado.cuotas.monto, ...

7. RECORD en sentencias DML

Cuando la variable es un %ROWTYPE de la tabla, Oracle permite insertar o actualizar la fila completa sin listar las columnas.

```
DECLARE
  v_filas CLIENTE_TODOSUMA%ROWTYPE;
BEGIN
  v_filas.nro_cliente      := 3;
  v_filas.run_cliente      := '16044255-7';
  v_filas.nombre_cliente   := 'STEPHANIE DIAZ KOHNERT';
  v_filas.tipo_cliente     := 'Trabajadores dependientes';
  v_filas.monto_solic_creditos := 540000;
  v_filas.monto_pesos_todosuma := 5400;

  INSERT INTO CLIENTE_TODOSUMA VALUES v_filas;  -- sin paréntesis
  COMMIT;
END;
/
```

Fíjate en el detalle: se escribe **VALUES v_filas**, sin paréntesis. Escribir **VALUES (v_filas)** es un error. Para actualizar existe la variante equivalente **SET ROW = v_filas**.

8. Errores frecuentes

Error de Oracle	Causa habitual	Cómo se corrige
PLS-00320 / PLS-00201	Se usó el nombre del tipo como si fuera la variable (t_ficha.nombre)	Usar el nombre de la variable: v_ficha.nombre
ORA-06502	El SELECT ... INTO tiene las columnas en distinto orden que los campos del RECORD	Alinear el orden. La carga es posicional
PLS-00306	Se intentó imprimir o concatenar el RECORD completo	Un RECORD no se convierte a texto: recorrer sus campos
PLS-00382	Asignación entre dos RECORD de tipos diferentes	Usar el mismo tipo, o asignar campo por campo
PLS-00218	Campo NOT NULL sin valor por defecto	Agregar := valor en la declaración del campo
ORA-01403 (no data found)	El WHERE del SELECT ... INTO no devolvió filas	Verificar el filtro; más adelante, controlarlo con la excepción NO_DATA_FOUND
No se ve ninguna salida	Falta activar la salida de servidor	Ejecutar SET SERVEROUTPUT ON antes del bloque

9. Resumen para consultar rápido

Necesito...	Escribo...
Definir una estructura propia	TYPE t_x IS RECORD (campo tipo, ...);
Declarar la variable	v_x t_x;
Copiar la estructura de una tabla	v_x TABLA%ROWTYPE;
Copiar la estructura de un cursor	v_x c_cursor%ROWTYPE;
Copiar el tipo de una columna	v_dato TABLA.columna%TYPE;
Leer o escribir un campo	v_x.campo
Llegar a un campo anidado	v_x.subrecord.campo
Llenar el RECORD desde la BD	SELECT col1, col2 INTO v_x FROM ... (por posición)
Llenar un solo campo	SELECT col INTO v_x.campo FROM ...
Copiar un RECORD entero	v_a := v_b; (solo si son del mismo tipo)
Insertar la fila completa	INSERT INTO tabla VALUES v_x; (v_x debe ser %ROWTYPE)
Actualizar la fila completa	UPDATE tabla SET ROW = v_x WHERE ...

Antes de empezar la actividad: verifica que el modelo esté cargado ejecutando `SELECT COUNT(*) FROM CLIENTE;` Debe devolver 130 filas. Y no olvides `SET SERVEROUTPUT ON.`